

Key Differentiators

1. TFLite model compiler produces a massively parallel CNN implementation in pure Verilog code.
2. Enables AI processing of raw pixels at high resolution and frame rate.
3. Foundation vision models enable multiple downstream tasks from a single feature map.
4. Target FPGA for continuous training and deployment, or ASIC for large foundation models.

Potential Applications

1. [LAD-RCNN:A Powerful Tool for Livestock Face Detection and Normalization](#)
2. [EvConv: Fast CNN Inference on Event Camera Inputs For High-Speed Robot Perception](#)
3. [Sony Event-based Vision Sensor](#)
4. [CNN-based Methods for Object Recognition with High-Resolution Tactile Sensors](#)
5. [FusionRCNN: LiDAR-Camera Fusion for Two-stage 3D Object Detection](#)
6. [Low-complexity CNNs for Acoustic Scene Classification](#)
7. [Counting Varying Density Crowds Through Density Guided Adaptive Selection CNN and Transformer Estimation](#)
8. [Agricultural Plantation Classification using Transfer Learning Approach based on CNN](#)
9. [Predictive Modeling of Charge Levels for Battery Electric Vehicles using CNN EfficientNet and IGTD Algorithm](#)
10. [A CNN Approach for 5G mmWave Positioning Using Beamformed CSI Measurements](#)

Overview

State of the art convolutional neural network models (CNN) can be used to compress raw pixels into a feature map, which can then be used to drive various downstream tasks. These downstream tasks include computational vision (e.g. stereo correspondence, optical flow), object detection and segmentation, and image encoding into multimodal large language model (LLM) tokens [1-6].

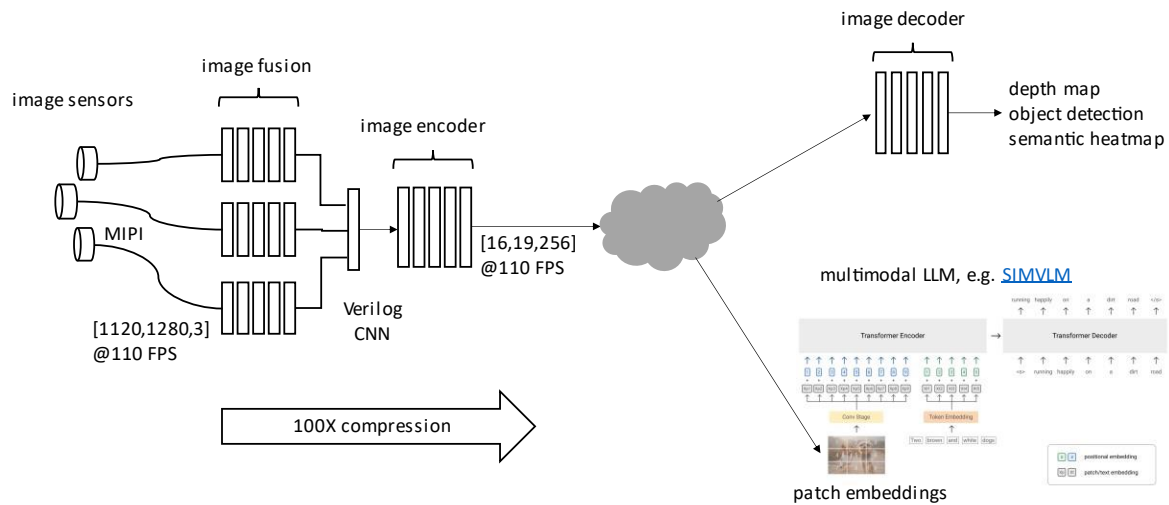


Figure 1

Current generation image sensors can generate high pixel rates e.g. $1600 \times 1400 @ 90 \text{fps} = 201 \text{M pixels/s}$ per camera. To efficiently evaluate a CNN at this data rate, we use pure Verilog code to achieve three degrees of parallel implementation. First, in each layer the dot products for all output channels are computed in parallel using dedicated multiply/accumulate (MAC) hardware. Second, the incoming tensors to each layer are evaluated in parallel using vertical stripes, again with dedicated hardware. Third, all layers are computed in parallel using pipelining. The effective throughput of the Verilog CNN is proportional to the product of these three terms, which can easily produce 10K+ MAC units which all run in parallel. To keep that many MAC units busy, we must eliminate memory bottlenecks by moving the model weights from monolithic, off chip memories to distributed, on chip memories.

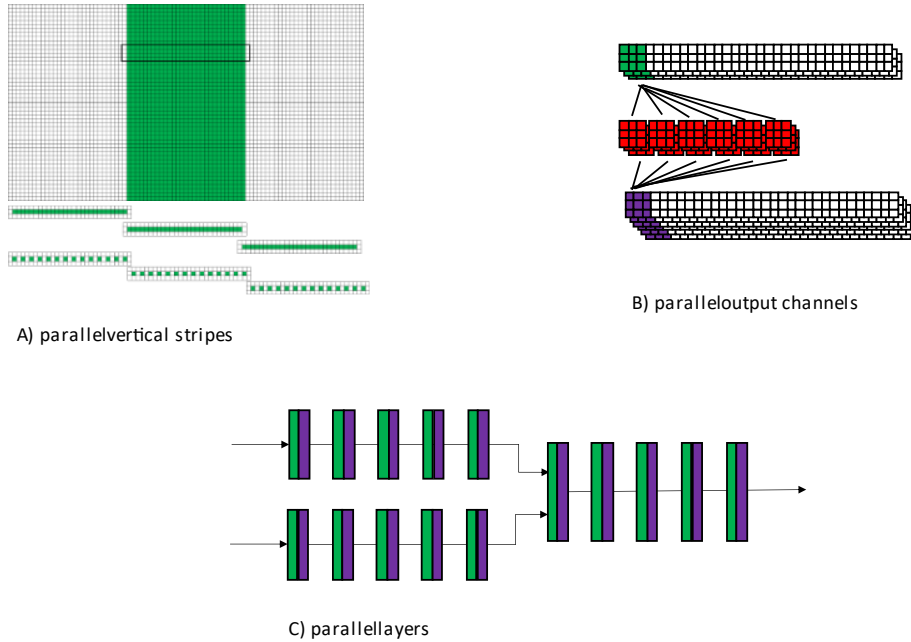


Figure 2

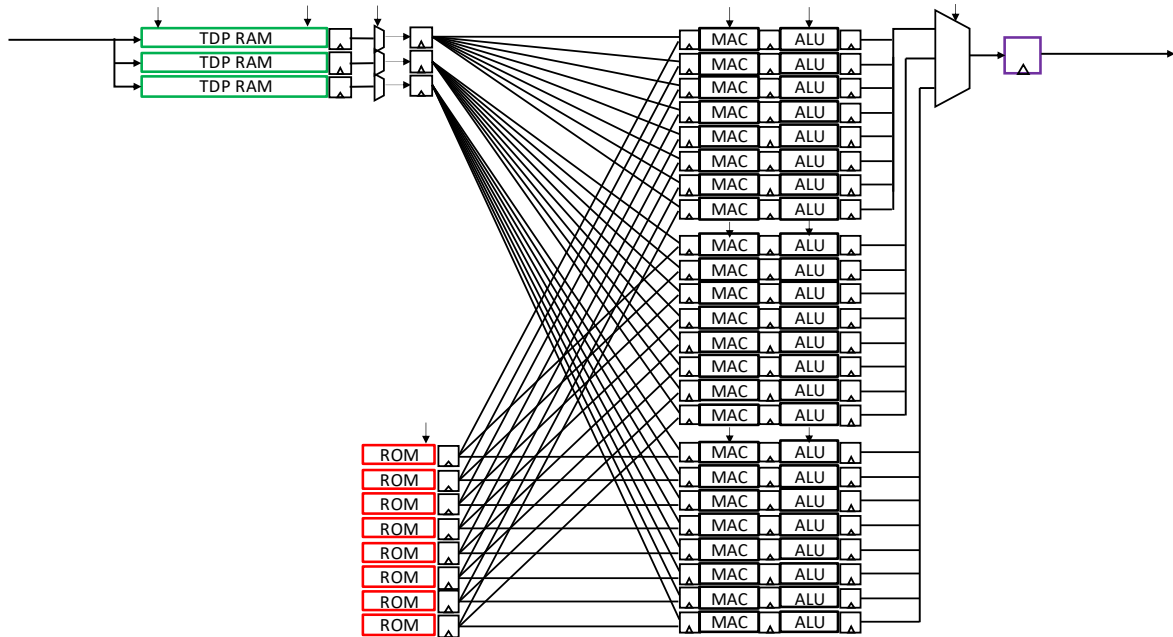


Figure 3

We have developed a compiler called Andromeda which converts a trained TFLite CNN model into pure Verilog code, using the parallel implementation described above. The compiler takes a target Verilog clock rate and image sensor row time as parameters, adjusting the generated Verilog code as needed to achieve real time performance. The generated Verilog code uses a streaming architecture which uses only row buffers and operates on a fixed schedule without flow control. This results in a solution with

very high throughput and low latency, with no external components. The generated Verilog code is a module level IP with streaming AXI-S input and output interfaces. This IP can be simulated, compiled to an FPGA bitstream, or hardened into an ASIC block.

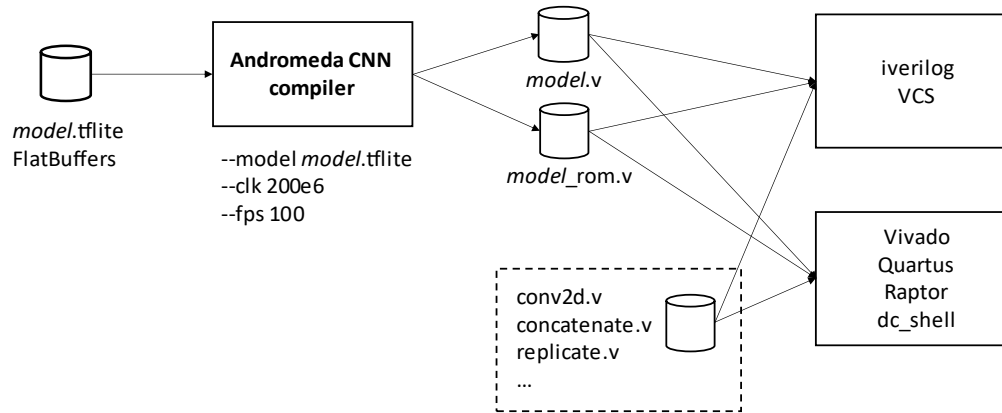


Figure 4

A typical application for this IP is a real time image encoder with the following benefits:

- 1) real time performance at maximum image sensor resolution and frame rate
- 2) supports fusion of multiple input frames into a single output feature map
- 3) can achieve >100X compression converting from raw pixels to features
- 4) can drive multiple downstream tasks, including traditional machine vision and multimodal LLM
- 5) larger models can be ASIC hardened using higher target clock rate and ROM based weights

Model training can follow different strategies. One approach is to use continuous learning with reconfigurable FPGA bitstreams, enabling the model to learn an unknown distribution over time. Another approach is to collect a large dataset and train a foundation model with static weights and hardened ASIC implementation. This will result in the highest performance and smallest area, for high volume applications.

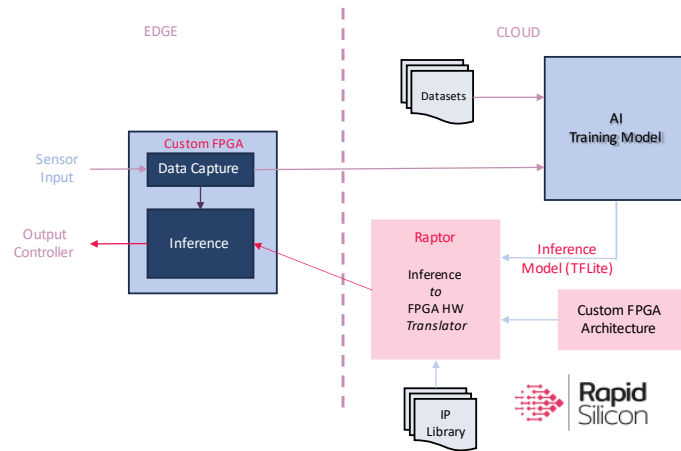


Figure 5

References

1	SIMVLM
2	A ConvNet for the 2020s
3	CLIP
4	Self-supervised Visual Feature Learning
5	AoCStream: All-on-Chip CNN Accelerator
6	GPU Utilization and CNN Inference

Datasheet

- Supports subset of TFLite operators (conv2d, concatenate, sum, replicate, ...)
- Conv2d supports arbitrary kernel size, stride={1,2}, activation={RELU, None}, padding={valid}
- Supports TFLite 8 bit integer weights with either 9b or 16b activations
- Supports striped execution for high speed conv2d
- Xilinx Artix US+ clock speed = ~200MHz
- Fusion Compiler area and clock rate for benchmark models, 16nm = TBD